

Sistemas de Tiempo Real
Departamento de Ingeniería Electrónica
Universidad de Antioquia
2026-2

Práctica No. 2
Cross-Compilation and Profiling
Análisis de desempeño de software

Realización: En parejas.

Valor: 10% del curso.

Fecha de entrega: Viernes 18 de septiembre del 2026.

1. Introducción

En el desarrollo de Sistemas de Tiempo Real no es suficiente verificar que un programa produzca resultados funcionalmente correctos. También es necesario estudiar su comportamiento computacional: tiempo de ejecución, utilización del procesador, llamadas entre funciones, utilización de memoria, accesos a memoria dinámica y posibles fugas de memoria.

El **profiling** permite obtener información dinámica sobre la ejecución de un programa y localizar las funciones o regiones que concentran una parte significativa del costo computacional. **Valgrind** proporciona diferentes herramientas para este propósito. Entre ellas, **Callgrind** permite realizar análisis basado en llamadas entre funciones y costos de ejecución; **Memcheck** permite detectar diversos errores relacionados con memoria; y **Massif** permite estudiar el uso del *heap* y, opcionalmente, de la memoria de pila.

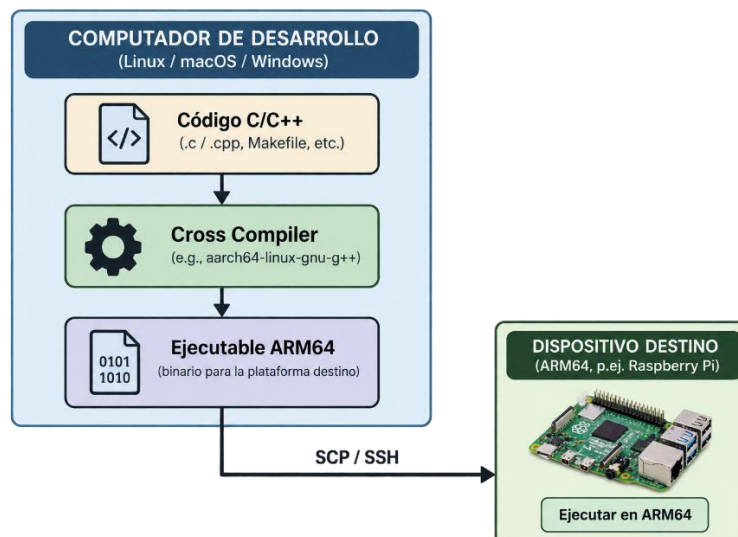


Figura 1: Flujo de Trabajo en su computador de desarrollo.

En esta práctica el análisis se realizará sobre una **Raspberry Pi 4**, que constituye la plataforma de ejecución *objetivo*. El código fuente será compilado inicialmente en un **computador de desarrollo** mediante un **compilador cruzado**, generando un ejecutable para la arquitectura ARM de la **Raspberry Pi**.

La **Figura 1** muestra el flujo de trabajo que será ejecutado en su **computador de desarrollo** (su portátil o PC de escritorio). Por otro lado, la **Figura 2** presenta el flujo de trabajo que será ejecutado en la **Raspberry Pi 4**.

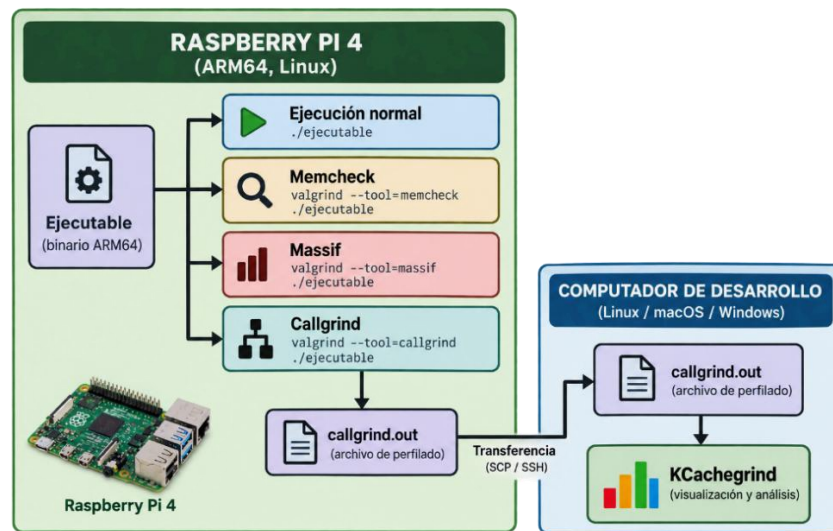


Figura 2: Flujo de Trabajo en la **Raspberry Pi 4**.

2. Preguntas orientadoras

2.1. Profiling:

- 2.1.1. Consulte y describa brevemente en qué consisten las herramientas **Valgrind**, **Kcachegrind** y **Massif**.
- 2.1.2. Consulte: ¿qué papel desempeña la creación de los *CallGraph* en el “*profiling*” de un determinado código fuente?
- 2.1.3. ¿Consulte, qué es un *hotspot* de ejecución?
- 2.1.4. En el contexto de *profiling*, ¿en qué consisten las fugas de memoria (*memory leaks*)?
- 2.1.5. ¿En qué consisten la *heap memory*, y la *stack memory*? Indique sus diferencias.
- 2.1.6. ¿En qué consisten los *snapshot* en la plataforma **Massif**?
- 2.1.7. ¿Por qué cree usted que el análisis de fugas y de uso de memoria es importante en el diseño de Sistemas de Tiempo-Real? (**Explique**).
- 2.1.8. ¿Por qué una herramienta de *profiling* puede modificar el comportamiento temporal del programa?

2.2. Compilación cruzada:

2.2.1. ¿Qué diferencia existe entre *compilación nativa* y *compilación cruzada*?

2.2.2. ¿Qué significa que un programa haya sido compilado para **ARM64**?

3. Arquitectura experimental

Como se mostró en la **Figuras 1** y **Figura 2**, en la presente práctica de laboratorio se utilizarán dos sistemas de cómputo diferentes:

3.1. Computador de desarrollo (PC de escritorio o portátil):

El cual será utilizado para:

- Edición del código fuente.
- Procedimiento de compilación cruzada.
- Análisis de los archivos generados por la herramienta **Callgrind**.
- Visualización de resultados con la herramienta **KCachegrind**.
- Visualización de resultados con la herramienta **Massif-visualizer**.
- Análisis de resultados.

3.2. Dispositivo Raspberry Pi 4:

La tarjeta **Raspberry Pi 4** utiliza un procesador basado en **ARMv8 Cortex-A72** y puede ejecutar Raspberry Pi OS de 64 bits. En la presente práctica de laboratorio este dispositivo será utilizado como:

- Plataforma objetivo.
- Plataforma de ejecución.
- Plataforma de *profiling* mediante la herramienta **Valgrind**.

4. Instalación de herramientas requeridas para la presente Práctica

4.1. En el computador de desarrollo:

En su dispositivo usado como “*computador de desarrollo*”, y considerando un ambiente Linux de su preferencia, realice la instalación de las siguientes herramientas:

4.1.1. Herramientas de compilación cruzada:

Ejecute los siguientes comandos en su distribución de Linux instalada:

```
sudo apt-get update
```

```
sudo apt-get install build-essential
```

A fin de utilizar posteriormente la **Raspberry Pi 4** como plataforma de objetivo, en su **computador de desarrollo** instale el siguiente compilador cruzado para la arquitectura **ARM** de 64 de bits:

```
sudo apt install gcc-aarch64-linux-gnu g++-aarch64-linux-gnu
```

4.1.2. Instalación del Kcachegrind y Massif-visualizer:

Para realizar la posterior visualización de los archivos de salida generados en el *profiling* que será ejecutado dentro de la Raspberry Pi, en su **computador de desarrollo** instale las siguientes herramientas:

```
sudo apt-get install Kcachegrind
```

```
sudo apt-get install massif-visualizer
```

4.2. En la Raspberry Pi 4:

4.2.1. Instalación del Valgrind:

En el **Terminal** del sistema operativo **Raspberry Pi OS** instalado dentro de su **Raspberry Pi**, ejecute el procedimiento indicado a continuación:

```
sudo apt-get update
```

```
sudo apt-get install build-essential valgrind
```

5. Organización del proyecto

5.1. Descarga y análisis del código fuente requerido en la presente Práctica:

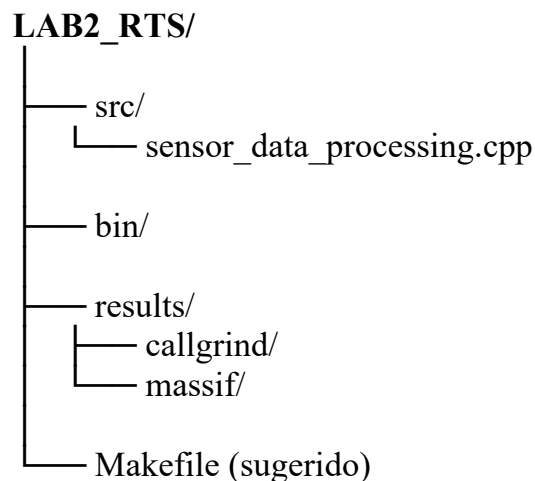
- 5.1.1. En la distribución Linux de su **computador de desarrollo**, cree la carpeta **LAB2_RTS**.
- 5.1.2. De la siguiente carpeta¹ del **Teams del curso**, descargue el código fuente llamado **sensor_data_processing.cpp**, y copie este archivo en su nueva carpeta **LAB2_RTS**.
- 5.1.3. Estudie y analice el código fuente suministrado, e indique sus aspectos más relevantes. Igualmente explique las palabras claves de C++ que para usted son desconocidas².

5.2. Creación de carpetas de trabajo:

Cree el siguiente conjunto de carpetas en su **computador de desarrollo**:

```
mkdir -p LAB2_RTS/src
mkdir -p LAB2_RTS/bin
mkdir -p LAB2_RTS/results
```

Tal que la estructura final de archivos sea la siguiente:



5.3. Compilación nativa:

- 5.3.1. Antes de realizar la compilación cruzada, compile una versión nativa del programa en el **computador de desarrollo**:

¹ <https://tinyurl.com/35jpekm3>

² Este código fuente está escrito en la versión C++ 11.

```
g++ -std=c++11 -ggdb3 -O0 -o bin/sensor_host \  
src/sensor_data_processing.cpp
```

La opción **-g/-ggdb3** incorpora información de depuración que permite relacionar los resultados del análisis con el código fuente. La documentación de **Valgrind** recomienda compilar con información de *debugging* y señala que **-O0** puede facilitar la interpretación de resultados, aunque con un costo de desempeño.

5.3.2. Ejecute ahora:

```
./bin/sensor_host
```

5.3.3. Verifique que el programa funcione correctamente.

5.4. Compilación cruzada:

5.4.1. Aun en el **Terminal** de su **computador de desarrollo** ejecute el siguiente comando:

```
aarch64-linux-gnu-g++ -std=c++11 -ggdb3 -O0 -Wall -Wextra -pedantic \  
-o bin/sensor_rpi src/sensor_data_processing.cpp
```

5.4.1. Consulte y describa brevemente qué hace cada una de las opciones indicadas en el comando anterior.

5.4.2. Verifique el ejecutable:

```
File bin/sensor_rpi
```

Debe identificarse como un ejecutable para **ARM/AArch64**.

5.4.3. Copie su archivo ejecutable en la **Raspberry Pi 4**, y allí verifique el funcionamiento este ejecutable:

```
./sensor_rpi
```

5.5. Preguntas:

5.5.1. ¿Qué diferencia existe entre **sensor_host** y **sensor_rpi**?

5.5.2. ¿Por qué el ejecutable **ARM** no puede ejecutarse directamente en un computador **x86-64**?

5.5.3. ¿Qué función cumplió entonces la herramienta **aarch64-linux-gnu-g++** utilizada anteriormente?

6. Análisis del Desempeño del código de su Aplicación

6.1. Instrumentalización de código fuente:

- 6.1.1. Consulte la utilidad y el uso de los macros de **Valgrind** denominados “CALLGRIND_START_INSTRUMENTATION”, “CALLGRIND_STOP_INSTRUMENTATION” y “CALLGRIND_TOGGLE_COLLECT”.
- 6.1.2. En su **computador de desarrollo**, modifique el código descargado, y agregue la línea de código `#include <Valgrind/callgrind.h>`, a fin de incluir los macros de **Valgrind** requeridos para instrumentalizar la generación de CallGraphs.
- 6.1.3. A partir de su estudio realizado anteriormente del código fuente descargado, determine una sección de dicho código que merezca un análisis detallado del tiempo de ejecución (ciclos, llamados a funciones, recursiones, etc.) y delimite dicha sección con los macros “CALLGRIND_START_INSTRUMENTATION” y “CALLGRIND_STOP_INSTRUMENTATION”.
- 6.1.4. **Pregunta:** ¿Por qué es conveniente delimitar la región de interés en lugar de instrumentar indiscriminadamente toda la ejecución?
- 6.1.5. Escriba comentarios alrededor de las modificaciones que usted realice en el código fuente suministrado, y de la sección o área de código que usted seleccionó para llevar a cabo el análisis requerido.
- 6.1.6. Realice nuevamente la **compilación cruzada** de su código fuente, y copie el nuevo ejecutable en la **Raspberry Pi 4**.

6.2. Generación del *Callgraph* de cada programa:

- 6.2.1. Genere el **CallGraph** del ejecutable haciendo uso de la herramienta **Callgrind** de la plataforma **Valgrind**. Para ello, en la **Raspberry Pi** ejecute la siguiente línea de comando:

```
Valgrind --tool=callgrind --dump-instr=yes --cache-sim=yes \  
--instr-atstart=no --callgrind-out-file=results/callgrind.out ./sensor_rpi
```

- 6.2.2. Consulte e indique qué hacen las opciones **--dump-instr=yes**, **--cache-sim=yes** y **--instr-atstart=no**, indicadas en el comando anterior.

6.3. Análisis mediante KCachegrind

6.3.1. Copie o transfiera desde su **Raspberry Pi** a su **computador de desarrollo**, el archivo **results/callgrind.out** generado anteriormente, y cópielo en su carpeta **LAB2_RTS/results**.

6.3.2. A fin de observar el “*profiling*” realizado sobre el programa **sensor_rpi**, utilice la herramienta **Kcachegrind**. Para ello ejecute el siguiente comando:

kcachegrind callgrind.out

6.3.3. Para conocer mayores detalles del uso de la herramienta **Kcachegrind** y la información allí presentada, observe el siguiente tutorial:

<https://www.youtube.com/watch?v=h-0HpCbIt3A>

6.3.4. En el entorno gráfico del **Kcachegrind** observe el contenido de la ventana inferior llamada “*Call Graph*”. Analice este grafo presentado.

- En su análisis, interprete los datos numéricos presentados sobre dicho grafo, e indique cuáles funciones descritas en el código fuente analizado presentan el mayor y el menor tiempo de ejecución, respectivamente.
- Identifique y explique el *hotspot* del código fuente analizado.
- En el menú desplegado con el *click* derecho de su mouse, seleccione la opción “**Export Graph**”, y guarde su grafo como una imagen **PNG**.

6.3.5. **Pregunta:** ¿El *hotspot* identificado mediante **KCachegrind** coincide con la región de código que usted había seleccionado antes de ejecutar **Callgrind**? Justifique.

6.3.6. Ahora, en la barra superior de herramientas del **Kcachegrind**, desactive la opción “*relative*”.

- Indique cuánto es el estimado total de ciclos de ejecución de cada una de las funciones presentes en su código analizado.

6.3.7. En el centro del **Kcachegrind**, en las pestañas superiores, seleccione la pestaña **Types**.

- Allí, observe el tiempo que su código utiliza en diversas etapas del procesamiento en CPU (**Instruction Fetch, Data Read Access, Cycle Estimation**, etc), y acceso a la memoria cache (**L1/LL Instr. Fetch Miss, L1/LL Data Read Miss, L1 Miss Sum**, etc).

- Analice estos valores, e indique si su código analizado presenta faltas de lectura y de escritura en la cache (*misses*) (**Explique**).

6.3.8. En su **Informe Final** de la Práctica presente el anterior análisis realizado.

7. Análisis de Fugas y Uso de Memoria en C++

7.1. Análisis de Fuga de Memoria

7.1.1. Uno de los aspectos importantes del *profiling* de un determinado código fuente, es el análisis de sus posibles fugas de memoria debidas a fallas en la creación y manipulación dinámica de reservas de memoria. A fin de analizar el manejo de memoria en el código descargado para la presente práctica, ejecute ahora el siguiente comando en la **Raspberry Pi 4**:

valgrind --leak-check=full ./sensor_rpi

7.1.2. Consulte e indique qué hace la opción **-leak-check=full** indicada en el comando anterior.

7.1.3. Explique la salida obtenida luego de ejecutar el anterior comando. ¿Qué fallas de memoria presenta el programa analizado? (**Explique**).

7.1.4. Resuelva las eventuales fallas de memoria detectadas, y escriba comentarios y explicaciones de las fallas presentadas, y de las correcciones llevadas a cabo en el código fuente analizado.

7.1.5. **Compile** o **cross-compile** nuevamente su programa analizado siguiendo el comando indicado en la **Sección 5.3 o 5.4**. Dado que en este programa usted resuelve las fallas de memoria observadas anteriormente, nombre el nuevo archivo ejecutable como **sensor_rpi_corregido**.

7.1.6. Verifique que las eventuales fugas de memoria en efecto fueron resueltas.

7.2. Análisis de Uso de Memoria

7.2.1. Así como la ejecución de un programa o una aplicación puede ser optimizada luego de identificar las fugas de memoria presentadas, también resulta importante conocer el uso dinámico de memoria durante el tiempo que se ejecuta una determinada aplicación. A fin de analizar este uso dinámico de memoria en el código fuente analizado en esta Práctica, en la **Raspberry Pi** ejecute el siguiente comando sobre el primer programa ejecutable **sensor_rpi**:

**valgrind --tool=massif --peak-inaccuracy=0.0 --stacks=yes --heap=yes
--time-unit=ms --detailed-freq=1 --threshold=0.0 ./sensor_rpi**

- Observe el nombre del archivo **massif.out.XXXX** que ha sido generado con el comando anterior³.
- 7.2.2. Consulte e indique qué hacen las opciones **--stacks=yes** y **--heap=yes**, indicadas en el comando anterior.
- 7.2.3. Repita el procedimiento anterior para su archivo ejecutable **sensor_rpi_corregido**.
- Nuevamente observe el nuevo archivo **massif.out.XXXX** que ha sido generado con el comando anterior.
- 7.2.4. Copie o transfiera los dos archivos **massif.out.XXXX** de su **Raspberry Pi** a su **computador de desarrollo**, y copie dichos archivos en su carpeta **LAB2_RTS/results**,
- 7.2.5. A fin de observar el uso de memoria presentado en cada programa ejecutable analizado, utilice la herramienta **Massif-Visualizer**. Para ello, en el **Terminal** de su **computador de desarrollo**, ejecute el siguiente comando para el primer archivo de salida:

massif-visualizer massif.out.XXXX

- 7.2.6. Presente capturas de pantalla de los gráficos de memoria generados.
- 7.2.7. Analice los resultados obtenidos, los picos de memoria presentados y los instantes de tiempo asociados a cada pico de memoria.
- Específicamente, indique el instante de tiempo en que se presenta el pico de memoria. Para mayor precisión en la medida de este tiempo, puede ayudarse del comando **ms_print** (**consulte**).
- 7.2.8. Considere ahora el análisis de memoria de su versión modificada del código fuente, donde corrigió y eliminó las fugas de memoria encontradas en la **Sección 7.1.4** de esta guía.
- Dado este análisis realizado en el programa corregido, indique el instante de tiempo en el cual se libera la memoria.
- 7.2.9. En su **Informe Final** de esta Práctica presente el anterior análisis realizado.

³ El número **XXXX** indicado en su archivo de salida, corresponde al número de identificación (**PID**) del respectivo proceso dentro de su distribución Linux.

Informe

1. Explique la relevancia del trabajo realizado en esta **Prácticas No. 2** para el diseño e implementación de **Sistemas de Tiempo-Real**.
2. Presente en su informe las respuestas a las preguntas realizadas en la presente guía de laboratorio.
3. Igualmente, agregue las diferentes capturas gráficas pedidas, así como su explicación e interpretación de cada resultado observado.
4. Adjunte a este informe un archivo **zip** de su carpeta **LAB2_RTS**, incluyendo los diferentes archivos de código fuente analizados en esta Práctica de Laboratorio.
5. Si usó IA en la presente práctica de laboratorio, agregue una **declaración formal**⁴ del uso de IA (Vea [aquí un ejemplo](#) de esta declaración).
6. Presente conclusiones y bibliografía adicional utilizada para realizar este informe.

Referencias Bibliográfica

- How to profile a C program with Valgrind/Callgrind. Available on: <https://medium.com/@jacksonbelizario/profiling-a-c-program-with-valgrind-callgrind-b41f15b31527>
- Profiling with Valgrind and visualization with KCachegrind. Available on: <http://schellcode.github.io/profiling-and-visualization>
- Measuring code performance. CERN. Available on: <https://indico.cern.ch/event/561981/contributions/2579523/attachments/1462475/2259348/measure-perf.pdf>
- Paul Floyd. Valgrind Part 4: Cachegrind and Callgrind. Available on: https://accu.org/journals/overload/20/111/floyd_1886/
- Nate Hardison and Julie Zelenski. Valgrind Callgrind. Available on: <https://web.stanford.edu/class/archive/cs/cs107/cs107.1212/resources/callgrind>
- Paul Floyd. Valgrind Part 5 – Massif. Available on: https://accu.org/journals/overload/20/112/floyd_1884/
- Stephane Carrez. Optimization with Valgrind Massif and Cachegrind. Available on: <https://blog.vacs.fr/vacs/blogs/post.html?post=2013/03/02/Optimization-with-Valgrind-Massif-and-Cachegrind>
- Massif: a heap profiler. Available on: <https://valgrind.org/docs/manual/ms-manual.html>
- Analysis of modules with massif. Available on: <https://github.com/wazuh/wazuh/issues/9873>

⁴ Declaración de uso de IA: <https://revistas.ucasal.edu.ar/index.php/CI/ia>